

#04 · DAG 並行啟用

[修復] 技術代號 A-PO-3 · 約 0.5-1 天

一句話：可同時做的事被排隊

為什麼要修

「同時做多件事」的拓樸排序程式碼已寫好（Kahn 演算法），但主流程沒呼叫它。

現在怎樣

shared/orb-dag-scheduler.ts 已有 topologicalBatches() 與 runBatchesWithConcurrency()，但 orbTaskChainRunner 仍用 for-loop 順序執行。

問題在哪

明明可平行（音樂、旁白、封面同時生）卻一條一條跑，跨模態工作流時程多花 40%。

要改什麼

把 orbTaskChainRunner 主迴圈改成呼叫 scheduler 拿批次列表，每批跑 Promise.allSettled（同 path 已由 scheduler 自動序列化）。

涉及檔案

server/services/orbTaskChainRunner.ts（主 step loop 改寫）

server/services/orbTaskOrchestrator.ts（傳入 scheduler 配置）

server/services/agentCollaborationOrchestrator.ts:startCollaboration（session 建立後排程）

shared/orb-dag-scheduler.ts（既有，不改）

Claude Code 提示詞 (複製貼上)

把下面這段話完整複製，貼到 Claude Code (或 /claude 對話)。Claude Code 會依此完成此項修復。完成後跑下方驗證步驟確認。

請依照計畫附錄 A.4 實作 A-PO-3 DAG 並行真正啟用：

前置建議：A-P1-4 補償鏈 (白話版第 17 條) 尚未做時，並行模式失敗時無法乾淨退場。本任務可先做，但測試時要驗證失敗 fallback 安全。

1. 修改 server/services/orbTaskChainRunner.ts，把目前 for (const step of task.steps) 的順序迴圈改成：

```
```typescript
import { topologicalBatches, runBatchesWithConcurrency } from "../../shared/orb-dag-scheduler";

const dagSteps = task.steps.map((step, idx) => ({
 ...step,
 id: step.id ?? `step_${idx}`,
 path: step.pagePath,
 dependsOn: planData?.dependencies?.[step.id] ?? [],
}));

const schedule = topologicalBatches(dagSteps);
if (schedule.cycle) {
 logger.error("step_cycle_detected", { taskId, cycle: schedule.cycle });
 return executeSequentiallyFallback(task, ...); // 既有迴圈作 fallback
}

const concurrencyCap = Math.min(3, Math.max(1, Number(process.env.ORB_BATCH_CONCURRENCY ?? 3)));

const batchResult = await runBatchesWithConcurrency({
 batches: schedule.batches,
 concurrency: concurrencyCap,
 runStep: (step) => executeCurrentStepTools({
 task: { ...task, currentStepIndex: task.steps.findIndex((s) => s.id === step.id) },
 ...input,
 perStepToolResults: cumulativeResults,
 }),
});

if (batchResult.failed) {
```

```
return { outcome: "failed", reason: "batch_step_failed", stepsRun: batchResult.results.length };
}
...
```

2. 把既有的順序 for-loop 包裝成 `executeSequentiallyFallback()` 作為 cycle 偵測時的 fallback。

3. 修改 `server/services/agentCollaborationOrchestrator.ts:startCollaboration()` : session 建立後呼叫 `topologicalBatches()` 把結果存進 `session.sharedContext.scheduledBatches` , 並把 `implicitChainEdges` 寫入 audit log : `appendOrbAgentTaskAuditEvent(taskId, "scheduling.same_page_sequencing", `${e.from} → ${e.to}`)`。

4. 補 vitest 測試 `server/services/__tests__/orb-task-chain-runner-parallel.test.ts` :

- 4 步驟 plan (2 同 path、2 獨立) 應產出至少 2 批，同 path 不同批
- 5 步驟並行 (cap=3) 應觀察 inFlight 永不超過 3
- 觸發 cycle 應 fallback 到 sequential，不爆炸

不要做的事：不要重寫 `shared/orb-dag-scheduler.ts` (既有實作正確)。不要刪除 sequential fallback 路徑。不要把 concurrency 預設值改大於 3 (保護外部 API 配額)。

## 如何驗證做完了

跑 4 步驟 plan (2 條同 path、2 條獨立) → logger 看到  $\geq 2$  批；同 path 必在不同批。並行 5 步驟 (cap=3) → inFlight 永不超過 3。手動構造 cycle → 走 sequential fallback。

## 預估耗時

0.5-1 天